



Frontend Architecture & Product Documentation

Building an Intelligent Digital Identity Platform

Version 1.0

Author: Bhavya Patela

August 3, 2026

Table of Contents

Note: open this document in Microsoft Word and allow field updates (or press Ctrl+A, then F9) to populate page numbers in the table of contents.

1. Project Introduction

Kizuna is a personal password manager built around a simple conviction: the bond between a person and their secrets should never depend on trusting a third party.

1.1 What is Kizuna?

Kizuna is a modern, dark-first password manager. It stores login credentials, payment cards, secure notes, and identities inside encrypted containers called **vaults**, and layers intelligence on top: a force-directed **Identity Map** that visualizes your entire digital footprint, and an **AI Security Advisor** that reviews the vault and produces personalized, actionable recommendations. The name comes from the Japanese word 絆 (kizuna), meaning **bond** or **trust** — the emotional core of the product.

Today, Kizuna is a complete, production-quality frontend built with Next.js and React. Every screen, interaction, and data flow works end-to-end against an in-memory demo layer that is deliberately shaped like the future backend. When the FastAPI backend arrives, it slots in behind an existing service interface without any UI rewrites.

1.2 Why was it created?

Most people manage passwords in one of three bad ways: they reuse a small set of memorized passwords, they store them in notes apps and spreadsheets, or they rent space in a commercial password manager whose inner workings they cannot inspect. Each option trades away either security, convenience, or trust.

Kizuna was created to demonstrate — and eventually deliver — a fourth option: a password manager whose entire client is transparent, self-hostable, and designed from day one for **zero-knowledge encryption** (a design where the server can store your data but can never read it, because decryption keys never leave your device).

1.3 The problem it solves

- **Password fatigue** — the average person maintains 80–100 online accounts. No human memory scales to 100 unique strong passwords, so people reuse them, and a single breached site exposes everything.
- **Invisible risk** — even people who use password managers rarely know **which** accounts are weak, reused, stale, or over-connected. Kizuna surfaces that continuously instead of hiding it in a settings page.
- **Fragmented identity** — your accounts form a web: one email resets dozens of services. Kizuna's Identity Map makes that invisible web visible, which is the first step to securing it.

1.4 Why build one instead of using an existing manager?

Excellent commercial managers exist — 1Password and Bitwarden set a high bar. Kizuna is still worth building for four reasons. First, **ownership**: an open, self-hosted stack means no subscription, no

vendor lock-in, and no company between you and your secrets. Second, **product direction**: mainstream managers are lists; Kizuna treats your digital identity as a *graph* and adds an analyst layer on top — a fundamentally different product thesis. Third, **AI-readiness**: the architecture reserves clean seams for LLM-driven analysis that incumbent products are only beginning to explore. Fourth, honestly, **craft**: building a security product end-to-end — encryption model, UX, and intelligence — is the strongest possible demonstration of engineering range.

1.5 Vision and long-term mission

The vision is for Kizuna to graduate from “a place passwords live” to a **digital identity guardian**: software that knows the shape of your online life, watches it for risk, explains problems in plain language, and helps you fix them in one click — while remaining mathematically unable to read your data. The long-term mission is captured in three words that structure this document: **Trust** (zero-knowledge architecture), **Bond** (the product relationship — proactive, personal, calm), and **Intelligence** (an advisor, not a filing cabinet).

2. Project Goals

2.1 Product goals

1. Feel like a premium desktop application — Linear, Raycast, and 1Password quality — not an admin dashboard.
2. Make security **legible**: every risk state (weak, reused, stale) must be visible, explainable, and fixable.
3. Be proactive: the product reviews itself daily via the Security Advisor rather than waiting for the user to audit.
4. Remain fully functional as a frontend demo today, with zero UI rewrites needed when the backend lands.
5. Keep every architectural seam AI-ready without shipping any AI dependency prematurely.

2.2 What makes Kizuna different

Conventional manager	Kizuna
A searchable list of credentials	A living graph of your digital identity
A security score buried in settings	A daily analyst with named, actionable findings
Trust the vendor's word	Zero-knowledge by design; client fully inspectable
Reactive (you audit)	Proactive (it reviews, you approve)

2.3 The five pillars

- **Trust** — the product must be verifiably unable to betray you: end-to-end encryption, no tracking, no data mining.
- **Bond** — security software people **like** opening. Tone, motion, and craft matter as much as ciphers.
- **Digital Identity** — accounts are treated as one connected identity, not isolated rows; the Identity Map is this pillar made visible.
- **Security & Privacy** — client-side encryption, minimal metadata, and features (like the Advisor) that run analysis locally today.
- **Future AI** — a rule-based advisor today whose interface is exactly the contract a future LLM service will fulfill.

3. Technology Stack

Every dependency was chosen deliberately. This chapter explains each one in plain language: what it is, why it was chosen, and where Kizuna uses it.

3.1 Framework and language

Next.js 16 (App Router)

Next.js is a framework built on top of React that handles the hard infrastructure of a web application: routing (which URL shows which screen), code splitting (sending the browser only the JavaScript each page needs), server-side rendering (generating HTML before the browser runs any code, so first paint is fast), and build tooling. **Why chosen:** it removes an entire category of configuration work, its App Router gives every route its own automatically code-split bundle (critical for keeping heavy features like the 3D globe and Identity Map out of the initial download), and it is the industry-standard React framework, which matters for maintainability. **Where used:** the entire application shell — every page under `src/app/` is a Next.js route.

React 19

React is the user-interface library: you describe screens as reusable **components** (self-contained pieces like a button, a table, a dialog) and React efficiently updates the browser when data changes. **Why chosen:** the component model maps perfectly to a design-system-driven product, and its ecosystem (TanStack Query, Radix, Motion) is unmatched. **Where used:** every visual element in Kizuna is a React component.

TypeScript (strict mode)

TypeScript is JavaScript plus a type system — every function and object declares the shape of data it accepts and returns, and the compiler rejects mismatches **before the code ever runs**. **Why chosen:** in a security product, passing the wrong object into the wrong function is not a cosmetic bug. Strict typing also makes the codebase self-documenting: the `types/` folder is effectively the data dictionary for the future backend. **Where used:** every file; `services/` and `types/` form the API contract.

3.2 Styling and components

Tailwind CSS v4

Tailwind is a styling approach where small utility classes (`flex`, `rounded-2xl`, `text-muted-foreground`) are composed directly in markup instead of maintaining separate CSS files. **Why chosen:** it enforces consistency through design tokens (a fixed palette of colors, spacing, and radii declared once in `globals.css`), makes dark/light theming a token swap rather than a rewrite, and eliminates dead-CSS accumulation. **Where used:** all styling; the Kizuna palette (#09090B background, #111113 cards, #3B82F6 accent) lives as CSS variables consumed by Tailwind.

shadcn/ui + Radix UI

Radix provides unstyled, accessibility-complete primitives (dialogs, dropdowns, tabs) that handle the genuinely hard parts — focus trapping, keyboard navigation, screen-reader semantics. shadcn/ui generates styled components *into the codebase* (under `components/ui/`) rather than shipping a locked library. **Why chosen:** we own every component file and can restyle freely, while accessibility comes correct by default. **Where used:** the 40+ primitives in `components/ui/` — sidebar, command palette, sheets, dialogs, tables, accordion, and more.

Motion (Framer Motion) and GSAP

Motion is a React animation library used for interface animation — page entrances, card hovers, expanding recommendation cards, and elements animating away when completed. GSAP with ScrollTrigger drives the scroll-linked choreography of the landing page's 3D globe. **Why chosen:** Motion integrates natively with React's component lifecycle (essential for exit animations), while GSAP is the precision tool for scroll-driven timelines. Both are loaded only where used; GSAP ships exclusively inside the lazy globe chunk.

Lucide React

A consistent, tree-shakeable icon set (only icons actually imported are bundled). Used everywhere from the sidebar to severity indicators in the Security Advisor.

3.3 Data and state

TanStack Query v5

TanStack Query manages **server state** — data that lives somewhere else (vaults, items, recommendations) and is only cached in the browser. It handles fetching, caching, background refresh, loading/error states, and cache invalidation after writes. **Why chosen:** hand-rolling this with `useEffect` is the single most common source of bugs in React apps; Query makes the correct pattern the easy pattern. **Where used:** every read/write in `hooks/` (`useVaults`, `useAllItems`, `useRecommendations`, ...).

Zustand

A tiny global state store for **client-only UI state**: the current user profile, command-palette visibility, selected vault, and search text. **Why chosen:** it is a few kilobytes, has no boilerplate, and — critically — draws a hard line: secrets never enter it (see Chapter 8). **Where used:** `store/auth-store.ts`, `store/ui-store.ts`, `store/vault-store.ts`.

React Hook Form + Zod

React Hook Form manages form state with minimal re-rendering; Zod declares **schemas** (formal descriptions of valid data, e.g. "email must be a valid address; master password ≥ 12 characters") that validate input and produce TypeScript types automatically. **Why chosen:** every form gets identical validation, loading, disabled, and error behavior from one pattern, and the same Zod schemas mirror the validation the FastAPI backend will enforce with Pydantic. **Where used:** login, register, vault create/edit, item create/edit, settings.

3.4 Supporting libraries

Library	What it is	Where Kizuna uses it
Native Fetch	The browser's built-in HTTP client — no Axios needed	<code>lib/api-client.ts</code> wraps it once for all services
Sonner	Toast notifications (small transient confirmations)	Copy, save, login, advisor actions
next-themes	Dark/light theme switching without flash	Theme toggle; dark is the default
date-fns	Date formatting and math	"Updated 3 days ago", advisor staleness rules
clsx + tailwind-merge (cn)	Safely combine conditional CSS classes	Every component via <code>lib/utils.ts</code>
class-variance-authority	Declares component variants (size, tone)	Buttons, badges in <code>components/ui/</code>
three.js	3D rendering (WebGL)	Landing page Digital World globe
d3-force	Physics simulation for graphs	Identity Map layout

4. Frontend Folder Structure

The structure follows one rule: group by responsibility, not by file type alone — and keep every layer replaceable.

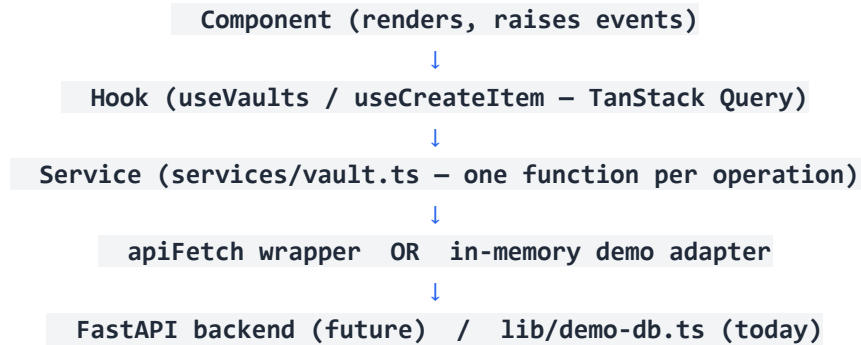
```
src/
├── app/                                # Routes (Next.js App Router)
│   ├── (auth)/                        # /login, /register – auth layout
│   ├── (dashboard)/                  # /dashboard, /vaults, /identity,
│   │   │                             # /advisor, /favorites, /generator, /settings
│   ├── layout.tsx                    # Root layout: fonts, providers, theme
│   └── page.tsx                      # Marketing landing page (/)
├── components/
│   ├── ui/                          # shadcn/Radix primitives (owned, restylable)
│   ├── landing/                     # Navbar, hero, mockups, 3D globe, FAQ
│   ├── auth/                        # Auth shell, forms, AuthGuard
│   ├── dashboard/                   # App sidebar, topbar, command palette, widgets
│   ├── vault/                      # Vault cards, item table, CRUD dialogs, generator
│   ├── identity/                   # Identity Map graph, drawer, insights
│   ├── security/                   # Security Advisor cards, timeline, stats
│   └── shared/                      # Logo, page header, empty state, copy button
├── hooks/                           # TanStack Query hooks + UI utilities
├── services/                        # API layer: auth, vault, folders, settings, advisor
├── store/                           # Zustand stores (UI state only – never secrets)
├── providers/                       # Theme, QueryClient, tooltip, toaster
├── lib/                             # api-client, password utils, validations,
│   │                             # identity graph builder, advisor rule engine, demo-db
├── types/                           # Shared TypeScript contracts (the data dictionary)
└── constants/                       # Navigation, vault icons, app config
```

4.1 Why each layer exists

- **app/** — thin route files only. Pages declare metadata and render a view component; they contain no logic, so routes stay trivially readable.
- **components/** — feature folders own their UI; **ui/** holds the generated design system; **shared/** holds cross-feature pieces. Nothing in components talks to the network directly.
- **hooks/** — the only place components get data. A component calls `useVaults()` and receives data plus loading/error state; it never knows where data came from.
- **services/** — the only place that knows *how* data is fetched. Today each function has a demo branch; tomorrow the same function calls FastAPI. Hooks and components are untouched by that swap.
- **store/** — deliberately small. If data could exist in a database, it does not belong here.
- **lib/** — pure logic with no React: the fetch wrapper, password entropy math, Zod schemas, the identity graph builder, and the advisor rule engine. Pure functions are the easiest code to test and to hand to a backend.

- **types/** — one source of truth for every data shape, shared by services, hooks, and components — and the exact contract the FastAPI models must satisfy.

4.2 How data flows



The one-way rule

Data flows down (service → hook → component) and events flow up (component → hook mutation → service). Components never import services directly, and services never import React. This single constraint is what makes the backend swap a non-event.

5. Overall Application Flow

A user's journey through Kizuna, from first visit to sign-out.



Alongside that spine, three destination features are reachable from the sidebar and the  command palette at any time: the **Identity Map** (</identity>), the **Security Advisor** (</advisor>), and the **Password Generator** (</generator>). The command palette also jumps directly to any vault or item.

5.1 Route protection

Every route inside the `(dashboard)` group is wrapped by `AuthGuard`, a client component that checks for an active session and redirects unauthenticated visitors to </login>. In the demo build the session is a localStorage flag; with the real backend this becomes a server-verified session cookie (Chapter 10). The landing page, login, and register remain public.

Route	Access	Purpose
/	Public	Marketing landing page with live product mockups
/login, /register	Public	Authentication (demo credentials shown on login)
/dashboard	Protected	Overview: stats, advisor widget, vault health, recent items
/vaults, /vaults/:id	Protected	Vault management and item CRUD
/identity	Protected	Force-directed digital identity graph
/advisor	Protected	AI Security Advisor recommendations

Route	Access	Purpose
/favorites, /generator	Protected	Starred items; password generation
/settings	Protected	Profile, security, appearance preferences

6. Complete Feature Documentation

Every feature, documented the same way: why it exists, how it behaves, how it works inside, and how the backend and AI will extend it.

6.1 Landing Page (/)

Purpose. Communicate trust and craft in the first five seconds and convert visitors to the demo.

Why it exists: a security product is bought on credibility; the landing page *is* the credibility argument. **User interaction:** a sticky navbar (blurs on scroll), a hero with an interactive product mockup whose sidebar genuinely filters its rows, an animated 3D “Digital World” globe, feature cards, a security-flow explainer, tabbed product previews, an FAQ accordion, and two CTAs (Get Started Free / Login).

Internally, the page is a server-rendered composition of client components under `components/landing/`. The globe deserves note: it is fully procedural (no downloaded 3D model or textures). A shader sphere, an atmosphere pass, ~6,000 land dots sampled from a 1° land bitmask generated at build time from Natural Earth data, seeded activity clusters, and one merged arc geometry — five draw calls total. GSAP ScrollTrigger fades it in, scrubs rotation on scroll, and eases rendering to a stop when off-screen; `prefers-reduced-motion` renders a single static frame. The entire `three.js` + GSAP payload lives in a lazy chunk that only downloads when the section approaches the viewport, so the marketing page's initial load is unaffected.

Future backend: none required — the page is static. **Future AI:** none planned; the globe's “encrypted activity” is intentionally illustrative. **Edge cases handled:** WebGL unavailable (CSS placeholder stays), reduced motion, tab hidden (rendering pauses), narrow screens (text stacks above the globe).

6.2 Authentication (Login / Register)

Purpose. Gate the vault behind a master password and create the session. **User interaction:** split-panel screens — brand narrative on the left, form on the right. Login has email + master password with visibility toggle; register adds name, a live password-strength meter, and confirmation. A “Try the demo” card displays the demo credentials with one-click fill.

Current implementation. Forms are React Hook Form + Zod (`lib/validations/auth.ts`). The demo accepts exactly one credential pair (`demo@kizuna.app` / `Kizuna@2026`); success stores a session flag in `localStorage` via `lib/demo-session.ts` and redirects to the dashboard. Every demo-only line is isolated and commented; `AuthGuard` protects the app shell; logout clears the flag and returns to the landing page.

Concern	Today (demo)	With FastAPI
Credential check	Constant comparison in the client	Argon2id hash verification server-side
Session	<code>localStorage</code> flag	HttpOnly Secure cookie (JWT + refresh)

Concern	Today (demo)	With FastAPI
Route guard	Client redirect in AuthGuard	Middleware verifying the session cookie
Master password	Never stored anywhere	Never sent raw — used client-side for key derivation (Chapter 14)

Expected endpoints: `POST /auth/register`, `POST /auth/login`, `POST /auth/logout`, `GET /auth/me`. **Validation:** email format, password ≥ 12 chars (mirrored by Pydantic). **Edge cases:** wrong credentials (inline toast, no field hints about which part failed), session expiry mid-use (401 $\rightarrow$ redirect with return-path), double-submit (buttons disable during flight). **Future AI:** none — authentication must stay boring.

6.3 Dashboard (/dashboard)

Purpose. Answer “how is my vault doing?” in one glance. **Contents:** a time-aware greeting; the **Security Insights widget** (count of open advisor recommendations plus the single highest-priority one, with a Review button deep-linking to `/advisor`); three stat cards (items, vaults, favorites); the **Vault Health card** (a 0–100 score computed client-side from password strength and reuse, with weak/reused counts); and **Recently Updated** (five most recent items with hover-copy).

Internally the dashboard is pure composition — every number derives from the same `useAllItems()` / `useVaults()` queries used elsewhere, so it is always consistent with vault pages and costs no extra requests (TanStack Query deduplicates). **Future backend:** an aggregated `GET /dashboard/summary` avoids shipping all items just to count them once vaults grow large. **Future AI:** the greeting line becomes a one-sentence LLM digest (“You fixed two risks this week; one reused password remains”).

6.4 Sidebar, Navigation & Command Palette

Purpose. Desktop-app navigation: everything reachable in one click or one keystroke. The sidebar (shadcn's sidebar primitive) shows the Kizuna wordmark, primary navigation (Dashboard, Vaults, Identity Map, Security Advisor, Favorites, Generator), a live vault list with item counts and a quick “new vault” action, Settings, and the user menu with **Lock & sign out**. It collapses to an icon rail on desktop and becomes a slide-over sheet on mobile.

The **command palette** (`⌘K` / `Ctrl+K`, built on `cmdk`) searches pages, vaults, and items in one field, with actions for generating a password and switching theme. Navigation lives in one constants file (`constants/nav.ts`), so sidebar and palette can never drift apart. **Breadcrumbs** in the topbar derive from the URL, resolving vault IDs to names from the query cache. **Edge cases:** vault list skeletons while loading; active-state matching for nested routes; palette works from any protected page.

6.5 Vaults (/vaults, /vaults/:id)

Purpose. Vaults are the organizational and (future) cryptographic containers — Personal, Work, Finance, Infrastructure. **User interaction:** a responsive card grid with icon, description, item count, and updated time; hover lifts; a context menu and kebab menu offer Open / Edit / Delete. Creating and editing use a dialog with an icon picker; deletion is an explicit confirmation dialog that states how many items will be destroyed.

Internally, vault detail composes the item table plus vault-level actions. All mutations flow through `useCreateVault` / `useUpdateVault` / `useDeleteVault`, which invalidate the vault and item caches so every count updates everywhere at once. **Expected endpoints:** `GET/POST /vaults`, `GET/PUT/DELETE /vaults/{id}`. **Backend validation:** name 1–40 chars, icon from a fixed enum, ownership checks on every operation. **Database:** `vaults` table with `user_id` foreign key; deleting a vault cascades to its items (Chapter 13). **Edge cases:** deleting the vault you are currently viewing (redirects to `/vaults`), empty-vault state with a first-item CTA. **Future AI:** suggested vault assignment for new items (“this looks like Finance”).

6.6 Password Table & Item Details

Purpose. The working surface of the product: every credential in a vault, scannable and safe. **User interaction:** a TanStack Table with sortable Name/Updated columns, a text filter, a favorite star per row, masked passwords with per-row reveal and copy (buttons appear on hover, reveal never persists), username copy, and a row menu for Edit / Delete. Add/Edit opens a right-hand **sheet** with the full form: type (login, card, note, identity — conditional fields), name, username, password with inline **Generate** and live strength meter, URL, and notes. Deletion confirms in an alert dialog.

Internally, columns are defined in a factory (`item-columns.tsx`) receiving callbacks from the page — the table renders any item list, which is how Favorites reuses it with an extra “vault” label column. **Expected endpoints:** `GET/POST /vaults/{id}/items`, `PUT/DELETE /items/{id}`. **Backend note:** with zero-knowledge encryption the server stores an opaque encrypted blob per item plus minimal metadata; the *server never validates password contents* because it never sees them (Chapter 14). **Edge cases:** copy on insecure clipboard contexts (toast explains), extremely long names (truncation with tooltips), notes-only items (no credential fields shown). **Future AI:** breach-watch annotations per row.

6.7 Password Generator (/generator)

Purpose. Make the strong choice the easy choice. **User interaction:** a large regenerating password display with copy; a length slider (8–64); toggles for upper/lower/digits/symbols; a live entropy-based strength meter. The same engine powers the inline Generate button in the item form.

Internally, generation uses `crypto.getRandomValues` (the browser’s cryptographically secure randomness), excludes visually ambiguous characters (l/I/1/O/0), and guarantees at least one character from each enabled set. Strength is an entropy estimate (`lib/password.ts`) used across the app — table badges, advisor rules, health score — so “weak” always means the same thing.

Backend: none — generation must remain client-side so secrets never transit the network. **Future AI:** site-specific policy awareness (“this bank rejects symbols”) via a curated ruleset, not an LLM.

6.8 Identity Map (/identity)

Purpose. The signature feature: answer “what does my online presence actually look like?” by drawing accounts as a living graph around you. **User interaction:** a force-directed canvas — the user orb at center, services orbiting with category-colored glows and real favicons. Hover raises a glass tooltip and highlights the node's neighbors; click opens a details drawer (copy credentials, strength, tags, open-in-vault); drag nodes with spring physics; wheel-zoom, drag-pan, double-click to re-fit. Eight category filter chips (Gaming, Finance, Work, Personal, Social, Development, Streaming, Shopping) and six **View by** relationship modes (Category, Vault, Email, Provider, Strength, Last updated) regroup both edges and spatial clusters. A search box dims non-matching nodes; an insights rail summarizes most-connected email, largest vault, weakest cluster, oldest account, top provider, and average strength.

Internally, rendering is a hand-rolled canvas layer over **d3-force** physics — chosen over React Flow because hundreds of glowing, favicon-composited nodes stay at 60 fps only on canvas. A pure builder (`lib/identity/graph.ts`) converts items into nodes/edges: every service links to the user (backbone), and within each group a hub-and-spoke pattern ($k-1$ edges, never $O(k^2)$) keeps dense vaults readable. Category classification is derived (domain → name → owning vault), requiring no schema change. Weak and reused passwords render as amber/rose risk rings driven by a `flags` array. Positions persist across mode switches so the graph morphs rather than resets; the whole feature loads as a lazy chunk.

Future backend: graph assembly can move server-side (`GET /identity/graph?mode=email`) once item counts grow; favicon proxying through the backend removes a third-party request. **Future AI:** the `flags` array is the contract — a model can mark nodes `compromised`, `duplicate`, or `suspicious` and the renderer already knows how to display them; risk propagation colors paths outward from a breached hub. **Edge cases:** zero matching filters (guided empty state), WebGL-less browsers (the page works; canvas 2D needs no GPU), favicon failures (monogram fallback).

6.9 AI Security Advisor (/advisor)

Purpose. Turn Kizuna from reactive storage into a proactive analyst that reviews the vault daily. **User interaction:** severity-filtered recommendation cards (Critical / Warning / Suggestion / Success) with live counts. Each card expands smoothly to a plain-language description, a highlighted **Suggested action**, an impact estimate (“+14 pts”), and three actions: **Review** (deep-link to the relevant screen), **Mark as completed** (card animates away, score toast fires, timeline gains a green entry), and **Ignore** (restorable). A **timeline** interleaves surfaced findings with completed improvements; an **analytics row** shows resolved count, improvement points, average password age, reused and weak counts, and a security-score trend chart.

Internally — the architectural heart: recommendations come from a local **rule engine** (`lib/advisor/analyze.ts`) that inspects real vault data, producing findings like “The same password

protects Netflix and Flipkart” or “kizuna@gmail.com protects 14 connected services — enable MFA.” The engine is invoked *only* inside the service layer's demo branch; hooks and UI consume a stable **AdvisorRecommendation** contract. Recommendation IDs are deterministic (rule + subject), so statuses survive re-analysis — a contract the backend must preserve. Status changes are optimistic mutations with rollback.

Future backend: `GET /advisor/recommendations`, `PATCH /advisor/recommendations/{id}`, `GET /advisor/stats` — delete the rule engine, keep everything else. **Future AI:** Chapter 12 details the LLM flow; the short version is that the model replaces *rule authorship*, not the UI contract.

Edge cases: empty states per filter, ignored items hidden but restorable, stats resilient to zero-password vaults.

6.10 Favorites, Search & Settings

Favorites (`/favorites`) reuses the item table across vaults, showing each row's home vault; the star toggle is an optimistic mutation. **Search** exists at three levels: the command palette (global, keyboard-first), per-table filters, and the Identity Map's graph search — all client-side today, with `GET /search?q=` as the future server contract (Chapter 11 covers encrypted-search options). **Settings** (`/settings`) has three tabs: Profile (name/email, avatar initials), Security (auto-lock interval, clipboard-clear interval — `GET/PUT /settings`), and Appearance (theme cards for dark/light/system, favicon toggle, compact mode). Theme switching uses next-themes with zero-flash class swapping; dark is the default brand experience.

6.11 Cross-cutting behaviors

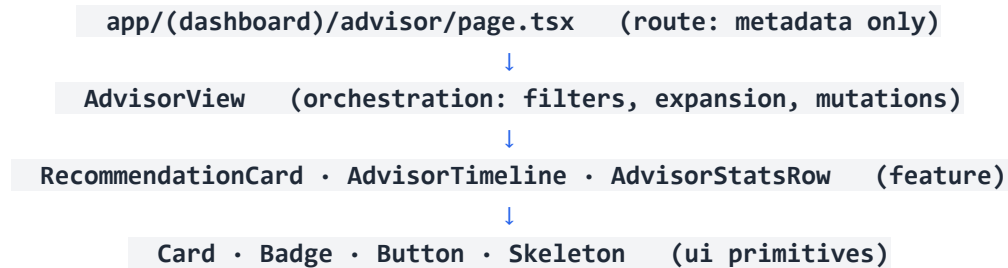
Behavior	Implementation
Notifications	Sonner toasts for every consequential action (saved, copied, deleted, signed out, advisor updates). One visual language, bottom-right, theme-aware.
Dialogs & sheets	Radix-based: dialogs for focused create/edit, sheets for rich item forms and drawers, alert-dialogs for destructive confirmation. All trap focus and restore it on close.
Forms & validation	One pattern everywhere: RHF + Zod resolver + shadcn Field primitives. Every form has loading, disabled, inline-error, and success states; submit buttons show spinners and lock during flight.
Responsive design	Desktop-first with deliberate mobile redesigns: sidebar → sheet, tables scroll within their container, grids collapse, the landing hero re-stacks. No horizontal page scroll anywhere.
Animations	Motion for interface (entrances, hover lifts, expand/collapse, exit animations); GSAP only for the landing globe. Everything respects prefers-reduced-motion.
Dark mode	Token-driven. Dark is default; light mode is a full first-class palette, not an inversion.

Behavior	Implementation
Loading states	Skeletons shaped like real content (sidebar rows, table rows, stat tiles, graph placeholder) — no spinner-only screens.
Error states	Query errors render inline retry affordances; mutations roll back optimistic updates and toast; the 404 page is designed, not default.
Demo authentication	Single accepted credential pair, session flag in localStorage, AuthGuard redirects, all isolated in lib/demo-session.ts with removal instructions inline.

7. Component Architecture

7.1 The layer model

Kizuna's components form a strict hierarchy. **UI primitives** (`components/ui/`) know nothing about passwords — a Button is a Button. **Shared components** (`components/shared/`) are Kizuna-flavored but feature-agnostic: `PageHeader`, `EmptyState`, `CopyButton`, `PasswordInput`, `PasswordStrength`, `Logo`. **Feature components** (`components/vault/`, `identity/`, `security/`, ...) compose the two layers below and receive data from hooks. **View components** (`*-view.tsx`) are the per-page orchestrators that own local UI state (filters, selection, open dialogs). **Page components** (`app/**/page.tsx`) only declare metadata and render a view.



7.2 Providers, hooks, utilities

Providers wrap the app once in `providers/app-providers.tsx`: `ThemeProvider` (next-themes), `QueryProvider` (a single `QueryClient` with 30-second stale time and one retry), `TooltipProvider`, and the `Sonner Toaster`. **Hooks** come in two families: data hooks (`use-vaults`, `use-vault-items`, `use-advisor`, `use-settings`, `use-auth`) that wrap `TanStack Query`, and UI hooks (`use-clipboard` with `feedback`, `use-debounce`, `use-mounted`, `use-mobile`, `use-password-generator`). **Utilities** in `lib/` are pure and React-free, which keeps the trickiest logic (entropy math, graph building, advisor rules) trivially unit-testable.

7.3 How everything communicates

- Components → hooks: the only way to read or write data. A component never fetches.
- Hooks → services: mutations call service functions and then invalidate the affected query keys (centralized in `lib/query-keys.ts`), so every consumer of that data refreshes automatically.
- Cross-page consistency is free: the sidebar count, dashboard stat, and identity graph all read the same cached `['items']` query.
- Cross-cutting UI state (palette open, selected vault) goes through `Zustand`, never through prop drilling.

Why this matters for the backend team

The frontend's entire data appetite is enumerable: `read services/` for every operation, `types/` for every shape, and `lib/query-keys.ts` for every cache relationship. Implementing `FastAPI` is filling in those

blanks — not reverse-engineering UI code.

8. State Management

Three kinds of state, three tools, one rule each. Choosing where data lives is the most consequential frontend decision — Kizuna makes it mechanical.

Kind of state	Tool	Examples	Rule of thumb
Server state	TanStack Query	Vaults, items, settings, recommendations, current user	If a database could own it, Query caches it
Global UI state	Zustand	Command palette open, selected vault id, search text, user profile snapshot	Cross-component, client-only, never secret
Local UI state	React useState	Dialog open, expanded card, reveal toggle, form fields (via RHF)	One component (or one view) cares

8.1 Why passwords are never in Zustand

Zustand stores are global, long-lived, and easy to subscribe to from anywhere — which is exactly why secrets are banned from them. A password that enters a global store can be read by any future component, lingers for the whole session, appears in devtools state dumps, and would survive route changes indefinitely. Instead, item data (including passwords) lives only in the TanStack Query cache — scoped, refreshable, cleared wholesale on logout via `queryClient.clear()` — and is **revealed** only transiently in component-local state that evaporates on unmount. The same discipline will apply post-encryption: decrypted values exist in memory just long enough to display or copy.

Logout is a state event

Signing out clears the Zustand user, wipes the entire query cache, and (today) removes the demo session flag. Nothing sensitive survives into the next session — a contract that carries over unchanged to the cookie-based backend.

9. API Architecture

9.1 One door to the network

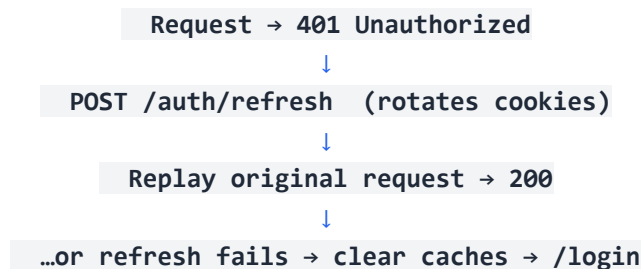
All HTTP goes through `apiFetch` (`lib/api-client.ts`), a thin wrapper over the browser's native Fetch API. It prefixes the base URL from `NEXT_PUBLIC_API_URL`, sends JSON, includes credentials (cookies), normalizes errors into a typed `ApiError` (parsing FastAPI's `detail` field), and handles empty 204 responses. Because there is exactly one door, adding auth headers, tracing, or retry policy later is a one-file change. When `NEXT_PUBLIC_API_URL` is unset, services transparently use the in-memory demo adapter instead — the UI cannot tell the difference.

9.2 What TanStack Query adds

- **Caching & deduplication** — five components asking for items produce one request and share one cache entry.
- **Staleness** — data is served instantly from cache, then refreshed in the background after its 30-second stale window.
- **Mutations & invalidation** — writes call a service, then invalidate exactly the query keys they affect (`lib/query-keys.ts` is the single registry), so dependent screens update themselves.
- **Optimistic updates** — favorites and advisor statuses apply instantly and roll back automatically if the server later refuses.
- **Loading & error states** — every hook exposes `isPending/isError`, which is why skeletons and retry affordances are consistent app-wide.
- **Retries** — one automatic retry on failure (never for mutations), a sane default for flaky networks.

9.3 Future authentication flow

With FastAPI, login sets two **HttpOnly cookies** (readable by the server only, invisible to JavaScript — the XSS-safe choice): a short-lived access token (~15 minutes) and a longer refresh token. `apiFetch` already sends cookies on every call. The planned refresh dance: on a 401, call `POST /auth/refresh` once, then replay the original request; if refresh also fails, clear all state and redirect to login. Session management stays server-authoritative — the frontend never inspects or stores tokens itself.



10. Backend Integration Guide

Written for the engineer opening the FastAPI repo on day one. The frontend is finished; this chapter is the contract it expects.

10.1 Ground rules

- Base URL comes from `NEXT_PUBLIC_API_URL`; all routes below are relative to it.
- JSON in, JSON out; `camelCase` field names to match `types/` exactly (configure Pydantic aliases).
- Errors: appropriate status + `{"detail": "human-readable message"}` — the client surfaces `detail` verbatim in toasts.
- Auth: HttpOnly session cookies; every non-auth route requires them and scopes queries by the authenticated `user_id` (authorization is `*ownership*`, checked on every row).
- Timestamps: ISO-8601 UTC strings.

10.2 Endpoint contract

Method & path	Purpose	Body (request)	Returns
POST /auth/register	Create account	name, email, password	user + cookies
POST /auth/login	Start session	email, password	user + cookies
POST /auth/refresh	Rotate tokens	—	204 + cookies
POST /auth/logout	End session	—	204
GET /auth/me	Session introspection	—	User
GET /vaults	List vaults (+itemCount)	—	Vault[]
POST /vaults	Create vault	name, description?, icon	Vault
GET /vaults/{id}	One vault	—	Vault
PUT /vaults/{id}	Update vault	partial VaultPayload	Vault
DELETE /vaults/{id}	Delete vault + items	—	204
GET /vaults/{id}/items	Items in a vault	—	VaultItem[]
GET /items	All items (identity/advisor/favorites)	—	VaultItem[]
POST /vaults/{id}/items	Create item	CreateVaultItemPayload	VaultItem
PUT /items/{id}	Update item (incl. favorite)	partial payload	VaultItem

Method & path	Purpose	Body (request)	Returns
DELETE /items/{id}	Delete item	—	204
GET /settings · PUT /settings	Preferences	partial UserSettings	UserSettings
GET /advisor/recommendations	Advisor findings	—	Recommendation[]
PATCH /advisor/recommendations/{id}	Review/ignore/complete	{ status }	Recommendation
GET /advisor/stats	Advisor analytics	—	AdvisorStats

10.3 Worked example: creating an item

Request — POST /vaults/vault_work/items

```
{
  "type": "login",
  "name": "Linear",
  "username": "kizuna@gmail.com",
  "password": "kV9#mTq2!xLp8&Zr",    // ciphertext blob once E2E encryption lands
  "url": "https://linear.app",
  "favorite": false
}
```

Response — 201 Created

```
{
  "id": "item_9f2c41ab",
  "vaultId": "vault_work",
  "type": "login",
  "name": "Linear",
  "username": "kizuna@gmail.com",
  "password": "kV9#mTq2!xLp8&Zr",
  "url": "https://linear.app",
  "favorite": false,
  "createdAt": "2026-08-03T14:21:09Z",
  "updatedAt": "2026-08-03T14:21:09Z"
}
```

Validation (Pydantic): name 1–60 chars required; type in the four-value enum; url must parse if present. **Errors:** 401 (no session), 403 (vault not owned by caller — return 404 to avoid existence leaks), 404 (vault missing), 422 (validation, with `detail`). **Database interaction:** insert into `items`, touch the parent vault's `updated_at`, write an `activity` row.

10.4 Suggested FastAPI shape

```
app/  
├─ routers/      auth.py · vaults.py · items.py · settings.py · advisor.py  
├─ services/     auth_service.py · vault_service.py · advisor_service.py  
├─ models/       SQLAlchemy: User, Vault, Item, Activity, Recommendation  
├─ schemas/      Pydantic mirrors of frontend types/ (camelCase aliases)  
├─ core/         security.py (Argon2, JWT), config.py, deps.py  
└─ main.py       CORS(allow_credentials, frontend origin), routers
```

The definition of done

Set `NEXT_PUBLIC_API_URL`, delete `lib/demo-db.ts`, `lib/demo-session.ts`, `lib/advisor/`, and the `!hasBackend()` branches, and the application must behave identically. That deletion list is the entire integration test.

11. Future Backend Roadmap

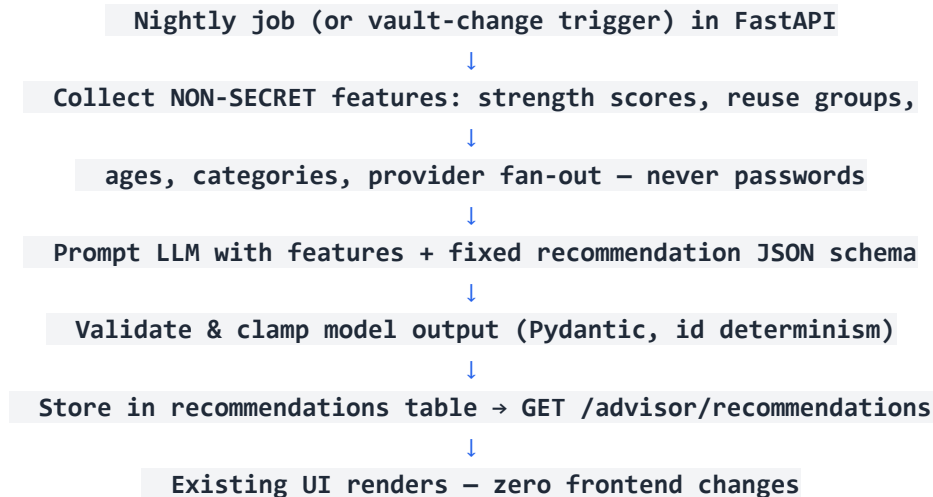
Capability	What the backend must provide
Authentication	Argon2id hashing, JWT + rotating refresh tokens in HttpOnly cookies, session revocation
Vault & item CRUD	Ownership-scoped endpoints per §10.2 with cascade rules and activity logging
Encryption & master password	Client-side key derivation (Argon2/PBKDF2) and per-item AES-GCM; server stores ciphertext + a verifier, never keys (Chapter 14)
Search	Server search over titles/URLs; under E2E encryption, either client-side search over synced indexes or searchable-encryption trade-offs — an explicit design decision for v2
Advisor	Persist recommendations/statuses; scheduled re-analysis; later the LLM pipeline (Chapter 12)
Identity graph	Optional server-side graph assembly for large vaults; favicon proxy
Activity & audit logs	Append-only event stream powering the advisor timeline and a future security log UI
Password history	Version rows on item update, enabling rollback and “rotated 3 times this year” insights
Import / export	Encrypted export; importers for Bitwarden/1Password/CSV with dry-run preview
Browser extension (future)	OAuth-style pairing, origin-matched autofill APIs, per-site item lookup

12. AI Integration Roadmap

12.1 The seam is already cut

Every AI feature lands behind an interface that already exists. The advisor consumes `AdvisorRecommendation[]`; the identity map renders `flags` on nodes; search accepts a query string. AI changes *how* those payloads are produced*, never how they are displayed — which is why the UI shipped first.

12.2 Planned LLM flow (Security Advisor)



Prompt engineering: a system prompt fixes tone (calm analyst, plain language), the output schema (the exact `AdvisorRecommendation` JSON), and hard rules (never invent account names; only reference provided features). **Background jobs** (Celery/APScheduler) keep latency out of the request path; results are **cached** per vault-state hash so unchanged vaults cost zero tokens. **Token budgeting:** features for even a 500-item vault fit in ~2–3k tokens; one nightly call per user is the ceiling. **Privacy is the non-negotiable:** the model sees derived metadata only — with E2E encryption the server *cannot* leak passwords to a model even by mistake, and a local-model deployment remains the self-host option.

12.3 Beyond the advisor

- **Identity insights** — LLM-written narrative for the graph (“your finance cluster depends entirely on one Gmail account”), rendered in the existing insights rail.
- **Natural-language search** — “streaming logins I haven't touched this year” → structured filter parameters (the model emits filters, never reads secrets).
- **Password recommendations** — policy-aware guidance; generation itself stays with the local CSPRNG.

- **Risk propagation** — graph traversal from a breached provider, surfaced through the already-supported node flags.

13. Database Planning

The schema the frontend implies. PostgreSQL assumed; UUID primary keys; every table carries created_at/updated_at.

Table	Key columns	Why it exists / relationships
users	id, email (unique), password_hash, name, master_verifier	Account root. Owns everything below.
vaults	id, user_id FK, name, icon, description	Organizational + future crypto boundary. users 1—N vaults.
items	id, vault_id FK, type, name, username, secret_ciphertext, url, notes, favorite, tags[]	The credentials. vaults 1—N items; cascade on vault delete.
item_history	id, item_id FK, secret_ciphertext, changed_at	Password history: one row per rotation; enables undo + age analytics.
tags / categories	join table or text[] on items	Free-form labels; identity categories stay *derived* client-side and need no table.
activity	id, user_id FK, item_id?, kind, metadata jsonb, at	Append-only audit log: feeds advisor timeline and future security log.
recommendations	id (deterministic), user_id FK, severity, category, title, description, action, impact, status, related_item_ids[], resolved_at	Advisor persistence; unique(user_id, id) upsert keeps statuses across re-analysis.
security_reports	id, user_id FK, score, stats jsonb, generated_at	Weekly snapshots → the trend chart's real data source.
identity_snapshots (optional)	id, user_id FK, graph jsonb, generated_at	Cached server-built graphs for very large vaults; purely an optimization.
sessions / refresh_tokens	id, user_id FK, token_hash, expires_at, revoked_at	Refresh-token rotation and remote sign-out.

Relationship summary: one user owns many vaults; each vault owns many items; items accumulate history rows and appear in activity events; recommendations and reports attach to the user. Nothing in the schema stores plaintext secrets once encryption lands — `secret_ciphertext` is opaque to every table that touches it.

14. Security Considerations

Security decisions explained simply — what protects the user, and why each choice was made.

14.1 The zero-knowledge model

The end-state design is **zero-knowledge**: everything sensitive is encrypted *on the user's device** before it is stored or transmitted, using keys derived from the **master password** — which itself never leaves the device. The server holds ciphertext it cannot open; a full database breach yields only noise. This is also why the frontend never invents its own “permanent” encryption today: half-done crypto in a demo would create false confidence. Encryption arrives as one deliberate layer with the backend: master password → key-derivation function (Argon2id — deliberately slow so guessing is expensive) → per-item AES-GCM encryption client-side → ciphertext up, ciphertext down.

What “the server can't read your vault” really means

Login proves you know the master password using a derived verifier — the password itself is never sent. The decryption key exists only in your browser's memory while the vault is unlocked. Forgetting the master password genuinely means the data is unrecoverable; that is the price and the point of zero-knowledge.

14.2 Web-layer protections

Threat / concern	Defense	In plain words
Eavesdropping	HTTPS everywhere	All traffic is encrypted in transit; cookies are marked Secure.
Token theft via XSS	HttpOnly cookies for JWT + refresh	Scripts cannot read the session, so injected code cannot steal it.
XSS itself	React escaping + no dangerouslySetInnerHTML + CSP (deploy-time)	User content renders as text, never as code.
CSRF	SameSite cookies + origin-checked CORS with credentials	Other sites cannot ride the user's session.
Credential stuffing	Rate limiting + Argon2id hashing (backend)	Guessing is throttled and each guess is computationally expensive.
Session lingering	Short access tokens, rotating refresh, auto-lock & clipboard-clear settings	Sessions expire fast; the UI already has the lock/clear preferences.
Shoulder surfing	Masked-by-default reveals, transient state	Passwords display only on explicit action and never persist revealed.

Two frontend habits complete the picture: secrets never enter global state or `localStorage` (Chapter 8), and logout wipes every cache. The demo session flag is the single, documented exception and is deleted with the backend swap.

15. Performance Considerations

15.1 Protecting Core Web Vitals

Core Web Vitals are Google's user-experience metrics: LCP (how fast the main content paints), INP (how fast the page responds to input), and CLS (how much the layout jumps). Kizuna's heaviest features are engineered to be invisible to them: the 3D globe and the Identity Map each live in **lazily loaded chunks** (dynamic imports behind Suspense) that download only when needed — three.js and GSAP never touch the landing page's initial bundle, and d3-force never touches any page but /identity. Fixed-aspect placeholders reserve layout space, so CLS stays at zero even while heavy content streams in.

15.2 Rendering and animation discipline

- Canvas over DOM for the two visualizations — hundreds of glowing nodes or dots are a handful of draw calls, not thousands of styled elements.
- Dirty-flag render loops: the Identity Map redraws only when physics, input, or data actually changed; the globe eases to a stop and pauses off-screen and on hidden tabs.
- Device-pixel-ratio caps (≤ 1.75 –2) keep 4K screens from quadrupling GPU work invisibly.
- Motion animates transform/opacity (compositor-friendly properties), and every animation path respects prefers-reduced-motion.
- Memoization where it pays: expensive derivations (graph builds, insights, advisor sorting) sit in useMemo; list callbacks in useCallback; and the React Compiler (enabled by the framework) removes most manual memo babysitting.
- TanStack Query caching means navigation between pages re-renders from warm data instantly instead of refetching.
- next/image optimizes the logo assets; fonts load via next/font with zero layout shift.

15.3 Accessibility as a performance feature

Accessibility shipped with the components, not after them: Radix primitives provide focus management and ARIA semantics; every interactive element is keyboard-reachable (including the command palette-first workflow); icons are aria-hidden with text labels; color never carries meaning alone (severity pairs color with icons and words); canvases carry role and label descriptions with equivalent controls in regular UI. The result is a product usable by screen reader — and faster for everyone, because semantic structure and visible focus benefit all users.

16. Developer Notes

16.1 Conventions that keep the codebase coherent

- **Files:** kebab-case (`item-form-sheet.tsx`); components export PascalCase; hooks start with `use`; one component per file, files stay small.
- **Folders:** feature-first under `components/`; never put feature logic in `ui/` (it is regenerable) — extend by composition in `shared/` or the feature folder.
- **Data:** new server data = type → service function → hook → component, in that order. Never fetch in a component; never invent a second query key for the same data (extend `lib/query-keys.ts`).
- **Forms:** RHF + Zod resolver + Field primitives, always — with loading, disabled, and error states from day one.
- **API:** all requests through `apiFetch`; demo behavior only behind `hasBackend()`; every demo branch commented with its replacement endpoint.
- **Styling:** design tokens only — no hex literals in components; spacing on the 8-pt grid; `cn()` for conditional classes.
- **Comments:** explain constraints and contracts, not narration; TypeScript types carry most documentation weight.

16.2 Practical reminders

- Demo data is in-memory: dev-server restarts reset vault changes and advisor statuses. This is intentional.
- Running `pnpm build` while `next dev` is running invalidates the dev cache — the next page load recompiles lazy chunks slowly. Restart dev after production builds to avoid confusing “empty” lazy sections.
- Regenerate the globe's land bitmask only when changing resolution: `node scripts/generate-land-mask.mjs`.
- Advisor recommendation IDs are deterministic on purpose — keep that contract or statuses will not survive re-analysis.
- Verify with `pnpm build` + `pnpm lint` (the single React-Compiler warning on TanStack Table is expected and benign).

16.3 Deployment recommendations

Deploy the frontend on Vercel (or any Node host) with `NEXT_PUBLIC_API_URL` pointing at FastAPI; serve both under one apex domain (`app.` / `api.`) so cookies flow cleanly with SameSite. Add a Content-Security-Policy at the edge, enable HSTS, and keep the FastAPI CORS allow-list to exact origins with credentials. CI should run build + lint on every PR; preview deployments make design review trivial.

17. Known Limitations

Area	Current state	Resolution path
Persistence	All vault data is in-memory demo data; resets on reload	FastAPI + PostgreSQL (Chapter 10)
Authentication	Single demo credential pair; localStorage session flag	Argon2 + HttpOnly JWT cookies
Encryption	None — demo stores plaintext in memory by design	Client-side E2E layer with the backend
Advisor intelligence	Local rule engine, deterministic	Server persistence, then LLM pipeline (Chapter 12)
Advisor statuses / trend	In-memory + seeded history	recommendations & security_reports tables
Search	Client-side filtering only	Server search; encrypted-search design in v2
Favicons	Third-party icon service (privacy trade-off), monogram fallback	Backend favicon proxy
Profile editing	Save is a stub toast	PUT /auth/profile
Folders	Service + types exist; no UI	Intentionally deferred until backend
Master password change / recovery kit	Not built	Requires real key-derivation flow first

Each limitation is a deliberate scope line, not an accident: everything above the service layer is production-shaped, and every mock is labeled in code with its replacement.

18. Future Vision — Kizuna 2.0

Version 2.0 is where the three pillars compound. The **Identity Graph** becomes temporal — watch your digital footprint grow, cluster, and (after cleanup) shrink. The **Security Advisor** becomes conversational: ask “what should I fix this weekend?” and get a ranked plan with one-click fixes; breach feeds trigger risk propagation across the graph in real time. A **browser extension** brings origin-matched autofill and inline generation; **desktop** (Tauri) and **mobile** (React Native) apps share the service contract, with biometric unlock and offline-first encrypted sync across devices. **Passkeys** join passwords as first-class items — Kizuna manages the transition away from passwords rather than pretending it isn't happening. **Team and shared vaults** introduce asymmetric key wrapping so families and teams share specific vaults without sharing master keys, opening the **enterprise** path: SSO, audit exports, and policy controls. Throughout, the invariant holds: every new surface consumes the same typed services, and nothing — not sync, not teams, not AI — ever requires the server to read a secret.

19. Conclusion

Kizuna today is a complete, premium frontend for an intelligent password manager: a marketing site with a genuinely novel 3D centerpiece; full vault and item management; a command-palette-driven shell; a force-directed Identity Map; and a Security Advisor that already behaves like the AI product it will become. Every screen ships with loading, empty, and error states; every form validates; every animation respects user preferences; dark and light themes are both first-class.

The architecture is **scalable** because heavy features are isolated in lazy chunks and pure libraries, **maintainable** because responsibility boundaries are strict and typed (routes → views → features → primitives; hooks → services → one fetch door), and **backend-ready** because the entire integration is an enumerable contract: the endpoints in Chapter 10, the schema in Chapter 13, and a deletion list of clearly labeled demo files. The FastAPI team does not need to read UI code to build the server — this document and the `types/` folder are the specification.

Most importantly, the product thesis is already visible in software: trust you can inspect, an identity you can see, and an advisor that works for you. The foundation is laid; what remains is to give it a backend worthy of it.

— *End of Document — Kizuna Frontend Architecture & Product Documentation, v1.0*